

Inside the Ear Canal: Four CVEs in the Softish C6 Camera and EarVision App

Cleartext video, a derivable Wi-Fi credential, unauthenticated device spoofing, and unsigned firmware — chained into full compromise of a consumer ear-canal camera

Matthew Dubbrin — Vexel Foundation

16 September 2026 · mdubbrin@gmail.com

ABSTRACT

The Softish C6 is a consumer ear-canal camera, sold under multiple white-label brand names and controlled by an Android application, EarVision, over a device-hosted Wi-Fi access point. I present four vulnerabilities discovered through static analysis of the companion application and passive wireless capture of the device's control and video protocol: cleartext transmission of live video (CVE-2026-81330), an access-point credential algorithmically derivable from a value broadcast in every beacon frame (CVE-2026-81640), acceptance of an unauthenticated rogue access point as the genuine device (CVE-2026-82563), and an unauthenticated, unsigned over-the-air firmware update path (CVE-2026-77974). Individually, each is a familiar class of IoT defect. Composed, they form a single passive-to-active chain: an attacker who only reads beacon frames can recover the Wi-Fi credential, decrypt the video feed, impersonate the device to the application, and — with one user tap — deliver an arbitrary firmware image with no signature check. I argue this chain is materially worse than the median IoT camera disclosure because the data in transit is medical-adjacent, frequently depicts a minor, and because no accountable vendor security contact could be established for the product. All four issues were coordinated through CISA under CSAF Advisory VA-26-251-01. Consistent with my exploit-code policy (§9), I withhold the credential derivation routine, rogue-access-point implementation, and video reconstruction tooling.

Keywords — IoT security, medical device privacy, cleartext transmission, derivable credentials, authentication bypass, unsigned firmware, coordinated disclosure

1 Introduction

Most consumer IoT camera disclosures end with "an attacker can watch your living room." This one ends with an attacker watching inside a body cavity, on hardware marketed for use on children, secured by a password that is broadcast in the clear ten times a second.

The Softish C6 is a handheld otoscope: a camera at the tip of a probe, a Wi-Fi radio in the handle, and no cloud service in between. A companion Android app, EarVision (package `com.atomath.wifi_camera`), streams the feed live to the user's phone. The same reference hardware ships under several storefront names — a typical arrangement for white-label IoT — which is itself a contributing factor in the disclosure difficulty documented in §7.

I treat this device as a case study rather than an isolated finding because the four vulnerabilities I identified do not fail independently. Each removes one control from the same trust boundary — transport, credential, identity, and integrity, in that order — and each failure makes the next one easier to reach. §5 formalizes this as a single attack graph. The result is that an attacker's cost to go from "in radio range" to "have pushed unsigned firmware to the device" is a passive capture, an off-the-shelf microcontroller, and one social-engineering-free tap on a prompt the legitimate update flow also produces.

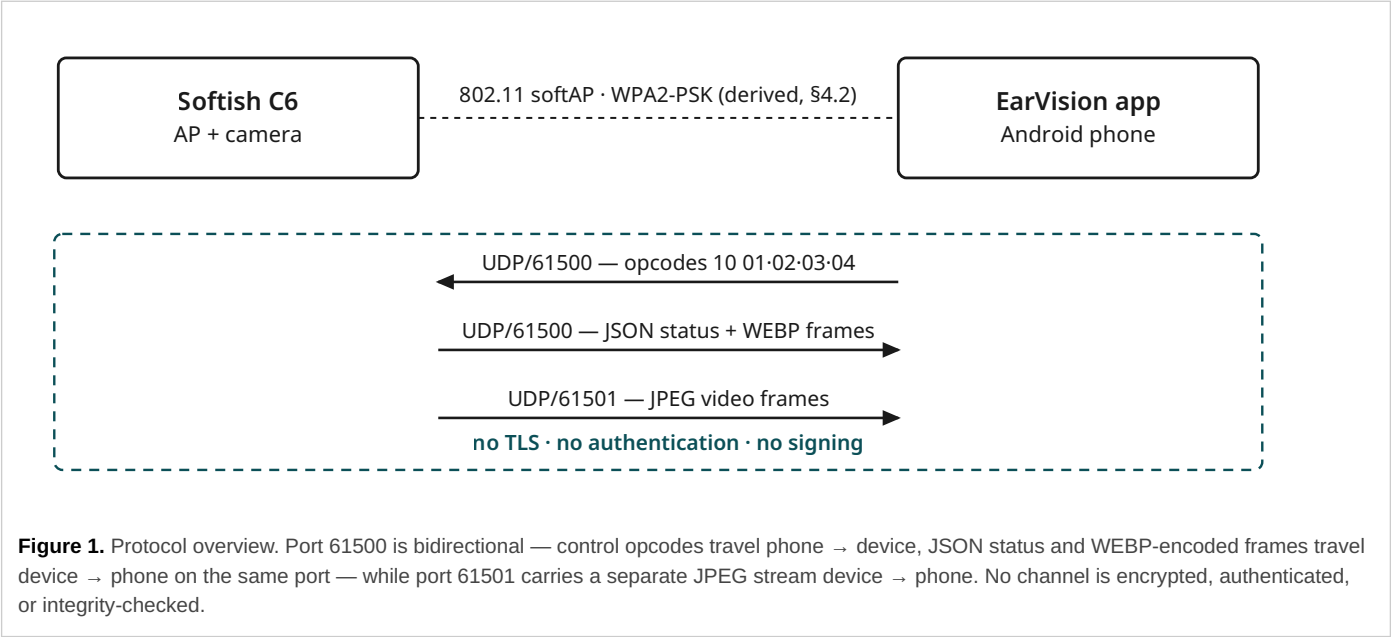
What distinguishes this device from a generic security-camera advisory is the sensitivity of the data in transit. The footage is ear-canal anatomy, eardrum condition, and any hearing-assistive hardware the subject wears; the product is marketed to parents as a way to check a child's ear at home rather than book a clinician visit. I return to this point

directly in §10, but it motivates the severity framing used throughout this paper: I treat confidentiality of the video channel as the anchor harm, not an afterthought to the firmware-integrity finding.

2 System Architecture & Threat Model

The C6 does not join the user's home network. It is its own access point: the phone disconnects from whatever network it was previously on, associates to the camera's SSID (of the

form `softish-XXXXXX`, where the suffix is derived from the device's BSSID), and the app speaks to the device directly over two UDP ports. There is no broker, no cloud relay, and — outside of the firmware images bundled in the APK itself — no external infrastructure involved in normal operation at all.



The application manifest declares this posture explicitly rather than leaving it implicit:

```
<application android:usesCleartextTraffic="true" ... >  
AndroidManifest.xml, com.atomath.wifi_camera v1.3.1
```

There is no TLS configuration to audit for weakness, because there is no TLS. The threat model I evaluate against is therefore the simplest possible one for a wireless device: an attacker with a commodity Wi-Fi adapter in monitor mode, within radio range of either the camera or the phone, with no prior credential and no physical access to either device.

Table 1 — Confirmed control opcodes, UDP/61500 (phone → device)

Opcode	Length	Function
10 01	2 B	Metadata poll — device responds with JSON status
10 02	2 B	Stream start request
10 03	4 B	Periodic metadata poll
10 04	786 B	OTA initiation — 16-byte header + raw firmware image (§4.4)

3 Methodology

All testing was performed against researcher-owned hardware in a researcher-controlled environment. No third-party device, network, or user was involved at any point.

- **Static analysis.** The EarVision APK was decompiled and its manifest, bundled assets, and Java/Flutter platform bridge reviewed. This is where `usesCleartextTraffic="true"`, the three bundled firmware images, the MNN inference plugin, and the bundled TFLite model (`core.db`, 5.2 MB, disguised with a non-standard extension) were located.
- **Wireless capture.** A wireless interface was placed in monitor mode and traffic between phone and camera captured across normal app usage — pairing, live viewing, still capture, and update checks.
- **Protocol analysis.** The two UDP channels were separated, the control opcodes on 61500 enumerated (Table 1), and the JSON status schema mapped field-by-field.

- **Frame reconstruction.** JPEG and WEBP frames were rebuilt from captured UDP payloads by locating frame boundaries and stripping the per-packet transport header, confirming the video is fully recoverable from a passive capture alone.
- **Spoofing proof of concept.** A rogue access point replicating the device's observed BLE and Wi-Fi identity was used to confirm the app accepts it as genuine, and that attacker-supplied version fields in the status JSON reliably produce the OTA prompt and subsequent firmware transfer.

4 Findings

Four CVEs were assigned, coordinated through CISA.^[1] They are presented below in the order used to brief the coordinating body — beginning with the passively-observable privacy failure, then the two findings that compose into a full man-in-the-middle position, then the finding that turns that position into arbitrary code execution.

Table 2 — Summary of assigned CVEs

CVE	Finding	CWE	CVSS 4.0
2026-81330	Cleartext live video over UDP	CWE-319	High · 7.1
2026-81640	AP password derivable from broadcast BSSID	CWE-798	High · 8.7
2026-82563	Device identity accepted from broadcast values, not cryptography	CWE-290	High · 8.4
2026-77974	Unauthenticated, unsigned OTA firmware transfer	CWE-306	High · 8.5

CWEs and CVSS v3.1 scores above come from CISA CSAF Advisory VA-26-251-01; v4.0 scores come from NVD's record for each CVE (same scoring source, `ics-cert@hq.dhs.gov`), since the CSAF advisory itself publishes v4.0 vectors without a computed base score. All four fall in the High band (7.0–8.9); none reach Critical (9.0+). CVE-2026-81640 carries the highest score of the four despite reading as the quieter, enabling step in the narrative below.

4.1 Cleartext Video Over UDP

CVE-2026-81330 High CVSS 4.0: 7.1 · CWE-319

Video leaves the device as ordinary JPEG frames on UDP/61501 (an 8-byte transport header followed by SOI/EOI-delimited JPEG data) and as WEBP frames interleaved with JSON control traffic on UDP/61500. Neither channel carries transport or application-layer encryption. Possession of the AP password — which, per §4.2, requires no cracking — is the only thing separating an observer from the video.

Frame reconstruction quality in my testing tracked capture quality directly: packet loss surfaced as dropped frames rather than failure. The attack degrades gracefully, which is the wrong property for a confidentiality control to have. Recovery is entirely passive — the camera cannot detect it, the app cannot detect it, and the user has no indication it is occurring.

4.2 Wi-Fi Password Derivable From a Public Value

CVE-2026-81640 High CVSS 4.0: 8.7 (highest of the four) · CWE-798

The C6's access-point password is unique per device rather than a single value shared across the product line — an implementation choice that, on its face, looks like the correct one. It is undermined by how that per-device value is produced: it is algorithmically derivable from the device's BSSID, which every C6 broadcasts in the clear, in every beacon frame, several times a second, to anyone listening. A value that is designed to be public cannot also function as the seed for a secret.

CWE-798 (use of hard-coded credentials) is the officially assigned weakness here: the credential is not literally hardcoded in firmware, but it is fully and deterministically derived from a broadcast value, which produces the same practical outcome. An attacker in range reads the BSSID off the air and computes the key — no handshake capture, no dictionary, no cracking time. CVE-2026-81330 follows immediately.

Consistent with §9, the derivation function itself is withheld.

4.3 Device Identity Accepted Without Cryptographic Proof

CVE-2026-82563 High CVSS 4.0: 8.4 · CWE-290

EarVision decides whether an endpoint is a genuine C6 using only broadcast identifiers — the Wi-Fi SSID and a BLE advertisement identity — rather than any cryptographic proof of possession. There is no device certificate, no challenge-response, no shared secret proved over the channel, and no signature on device responses. A microcontroller (an ESP32, roughly five US dollars) that replicates the expected BLE advertisement while hosting a matching SSID is accepted by the app as the real device, including auto-join, because the phone already holds the AP credential from a prior legitimate connection and §4.2 makes that credential trivial to reproduce independently as well.

Once accepted, the rogue endpoint controls every field of every JSON status response and every byte of every UDP/61500 reply — a full man-in-the-middle position with no further work required. This is the pivot into §4.4.

4.4 Unauthenticated, Unsigned Firmware Update

CVE-2026-77974 High CVSS 4.0: 8.5 · CWE-306

The C6 inverts the usual IoT firmware model: images are not fetched by the device from a vendor server, they are bundled inside the Android APK and pushed to the camera by the phone. Three images ship in the app, one per chipset variant (Table 3). The update decision is driven entirely by two device-reported fields in the status JSON:

Table 3 — Bundled firmware images (extracted from EarVision v1.3.1)

File	Size	Chipset	Notes
bk7252n_251117.bin	616 KB	Beken BK7252N	RBL container (magic 52 42 4C 00)
txw806_111.bin	368 KB	TXW806	version 111
txw810_125.bin	321 KB	TXW810	version 125

If ota_device_version is reported greater than device_version, EarVision presents an update prompt. Nothing authenticates either field, and — following §4.3 — both arrive over an unencrypted channel from an endpoint

the app never cryptographically verified. When the user accepts, the app transmits the bundled image over UDP/61500 as opcode 10 04:

Table 4 — Confirmed 10 04 OTA packet layout

Offset	Length	Value observed	Field
0x00	2 B	10 04	OTA opcode
0x02	4 B	7D 00 00 00	Current device version (125, little-endian)
0x06	4 B	10 E6 04 00	Firmware size in bytes (321,040)
0x0A	2 B	00 00	Padding
0x0C	4 B	00 00 00 03	Partition / chip identifier
0x10	770 B	<firmware>	Raw txw810_125.bin, byte 0 onward

UDP/61501 payload – offset 0x00

a9 de 52 fe 6e 57 01 a7 **ff d8** ff db 00 43 00 08 ...

| |
8-byte custom transport header JPEG SOI marker (FF D8)

Everything after the transport header is a standard JPEG stream:
no encryption, no obfuscation -- the SOI marker sits in the clear.

Figure 3. Confirmed payload structure on UDP/61501: an 8-byte custom transport header followed immediately by a JPEG SOI marker (FF D8, highlighted), sitting in the clear with no encryption or obfuscation.

I confirmed the transmitted payload is byte-for-byte identical to the bundled asset — not merely similar — by comparing the packet body against the extracted firmware file directly:

Table 5 — Byte-for-byte comparison, packet offset 0x10 vs. txw810_125.bin offset 0x00

Source	First 32 bytes
OTA packet (offset 0x10)	69 5A 00 1C 00 00 00 18 00 00 00 18 00 0C 00 00 ...
txw810_125.bin (offset 0x00)	69 5A 00 1C 00 00 00 18 00 00 00 18 00 0C 00 00 ...

No authentication guards the status channel that triggers the prompt — this is CWE-306, the officially assigned weakness for this CVE; no signature binds the image to the vendor, so code is downloaded and used with no integrity check; and the firmware header itself confirms integrity checking is disabled at the source, with `code_crc32 = 0x00000000` and `head_crc16 = 0x0000`. The device

acknowledged the transfer (`ota_success: true`) in its status response; confirming persistence across a power cycle would additionally require a physical flash dump, which was out of scope for this disclosure. The user-facing accept prompt is the only remaining gate, and it is visually identical to the prompt a legitimate update produces — the user cannot distinguish them because the app cannot.

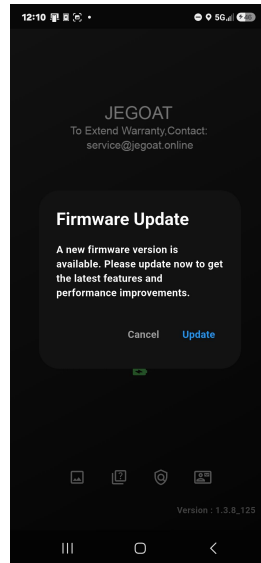


Figure 4. The in-app "Firmware Update" prompt, captured on two different firmware versions — identical and unverifiable either way.

5 The Composite Attack Chain

Individually, each finding above is a common IoT defect with numerous public precedents (§6). Composed, they form one continuous escalation from a completely passive

starting point to arbitrary code execution on the device, requiring exactly one moment of victim interaction at the final step.

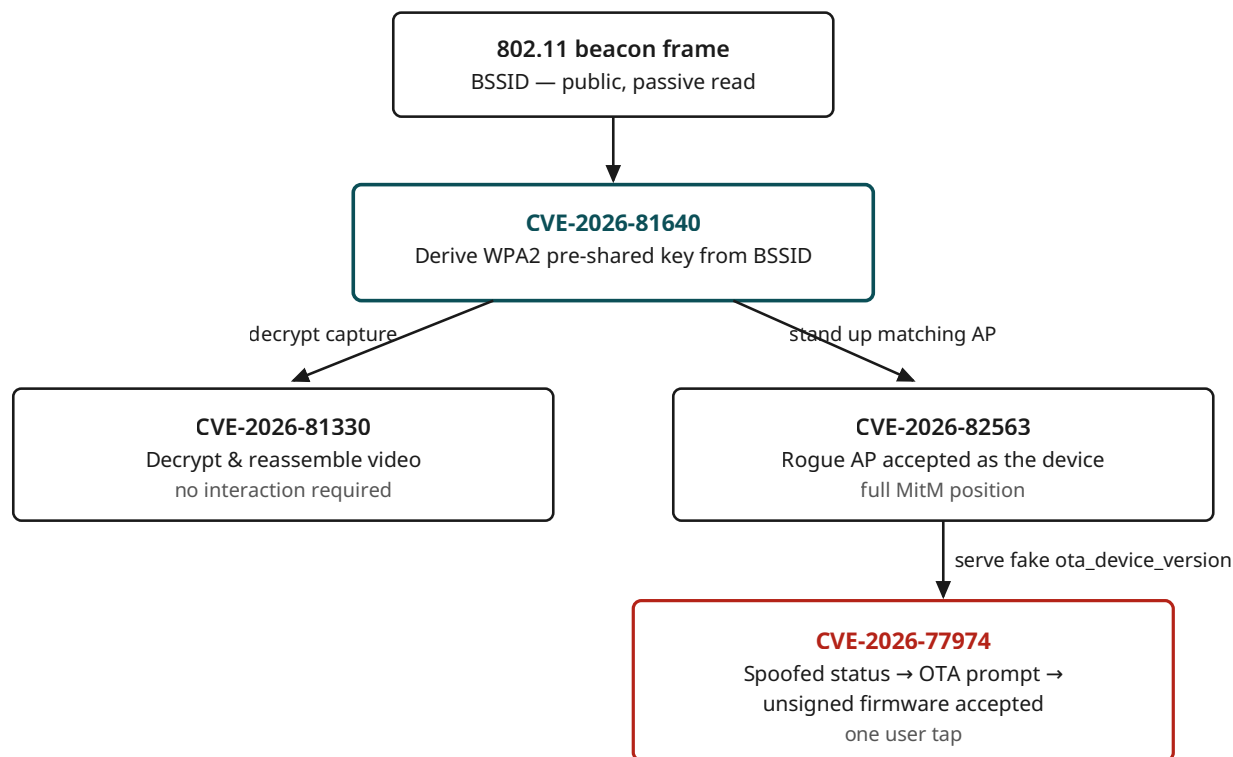


Figure 2. The four findings as a single attack graph. The left branch (video interception) needs no user interaction at any point; the right branch (identity spoofing) converges on firmware push, which needs exactly one tap on a prompt visually indistinguishable from a legitimate update.

Two properties of this chain are worth stating plainly. First, the step with the largest technical consequence — arbitrary firmware execution — requires the least technical sophistication of any step in the chain, because §4.2 and §4.3 do all the work of getting the attacker into position; the final step is a JSON field and a UDP send. This is a different measure than the CVSS ranking in Table 2: CVE-2026-81640 scores highest because it needs no user interaction and no prior access whatsoever, while CVE-2026-77974 is what actually hands the attacker persistent code execution on the device. Second, the video-interception branch (§4.1) needs no chain at all — it is

independently exploitable by anyone who captures the AP handshake or derives the password, with zero interaction with the app or its user.

6 Related Work

None of the four failure classes here are novel. Each has a well-known, independently disclosed precedent, several in categories more consequential than a novelty camera — which is precisely the point: this is not a research problem, it is a pattern that keeps recurring because nothing in the supply chain is accountable for fixing it (§10).

Table 6 — Comparable disclosed incidents by failure class

Incident	Failure class	Parallel to
Mirai botnet (2016) ^[8]	~62 hardcoded/derivable factory credentials on cheap DVRs and IP cameras	§4.2
Arlo cameras — predictable post-setup AP password ^[9]	Derivable/weak per-device Wi-Fi credential	§4.2
TP-Link Archer NX, CVE-2025-15517 ^[10]	Missing auth check permitting unauthorized firmware upload	§4.4
Wyze Cam, CVE-2019-9564 ^[11]	Auth-bypass camera takeover via spoofable device identity	§4.3
Hospira infusion pumps, CISA ICSA-15-125-01 ^[12]	Unauthenticated, unsigned firmware accepted by a medical device	§4.4
St. Jude / Abbott cardiac devices, FDA recall 2017 ^[13]	Weak/spoofable RF authentication on an implanted medical device	§4.3

The Hospira and St. Jude/Abbott cases are the closest analogs in consequence class rather than device class: both are medical devices where a missing authentication or integrity control on a wireless or network-reachable update path became a patient-safety issue rather than a privacy one. The C6 is not an infusion pump or a pacemaker, but it is marketed into the same "device that touches a body, built to

consumer cost targets" category, with none of the design scrutiny those categories eventually received after regulatory intervention.

7 Coordinated Disclosure Timeline

Table 7 — Disclosure timeline

Date	Event
May 2026	Initial discovery during static analysis of the EarVision APK
2026	Report submitted to CISA for coordination
2 Sep 2026	CVE-2026-81330, -81640, -82563, and -77974 published
8 Sep 2026	CISA publishes CSAF Advisory VA-26-251-01
16 Sep 2026	This paper published

I attempted to identify a vendor security contact before proceeding through CISA. The manufacturer of record, Shenzhen Jiding Electronics Co., Ltd., could be identified from the app's bundled privacy policy, but that document lists only a generic, personal-provider email address for privacy-policy questions — not a security contact, not tied to a published disclosure process, and not confidence-inspiring as a channel for a report of this severity. No `security.txt`, no dedicated vulnerability-disclosure page, and no security address appear on any storefront or documentation associated with the product or its white-label variants. This is the ordinary condition for this class of hardware, and it is the reason coordination proceeded through CISA rather than direct vendor contact.

8 Recommendations

8.1 For owners of this device

There is no patch to apply. The practical guidance is accordingly short:

- Treat any footage the device captures as though it were broadcast publicly, because in effect it is.
- Power the device on only for the duration of an actual use; the access point is live for as long as the device is on.
- Do not use it anywhere you would not accept a stranger in range watching the feed.

- Decline firmware update prompts unless you have specific reason to trust them — you have no way to verify one, and that is the point of §4.4.
- For anything that will inform a medical decision, particularly for a child, use a clinician instead of this device.

8.2 For vendors building this class of device

Every issue in this paper has a decades-old, well-understood remedy. None require novel research.

- **Encrypt the media channel.** DTLS or SRTP over the existing UDP transport; the frames are already framed, so wrapping them is not a redesign.
- **Authenticate the device cryptographically.** Provision a per-device key or certificate at manufacture and prove possession over the channel — an SSID is an identifier, not an authenticator.
- **Generate AP credentials from real entropy.** A random per-device password printed on the unit or its packaging is simultaneously more secure and easier to implement than any derivation function.
- **Sign firmware and verify on-device.** A bootloader check against a key burned into hardware at manufacture is the single control that would have neutralized §4.4 regardless of every other finding.
- **Do not trust device-reported state for security decisions.** An unauthenticated endpoint claiming it needs an update is not evidence that it does.
- **Publish a security contact.** A monitored address and a `security.txt` file cost nothing.

These recommendations track the baseline technical controls in NIST IR 8259A^[14] — device identification, secure configuration, data protection, and a signed software/firmware update mechanism — and the criteria behind the U.S. Cyber Trust Mark program defined in NIST IR 8425^[15], neither of which this product meets on any axis I tested. CISA's IoT acquisition guidance is a useful procurement-stage checklist for buyers of this device class more broadly.^[16]

9 Exploit-Code & Ethics Statement

WITHHELD BY POLICY

I am not publishing the BSSID-to-password derivation routine, the rogue access point implementation, or tooling to reconstruct video from a capture file. The affected behavior is described in this paper in enough technical detail for defenders, researchers, and anyone in the supply chain to understand and act on it, without shipping a working attack toolkit against an unpatchable device that has an unknown but non-trivial number of units already in homes. There is no automatic update path and no vendor security contact through which a fix could be distributed even if one existed; publishing working tooling in that position would harm current owners without meaningfully advancing anyone's understanding beyond what §4 already provides. Researchers who require additional technical detail for defensive or coordination purposes may request it by contacting me directly at mdubbrin@gmail.com.

10 Discussion: A Broader Pattern

Cheap white-label IoT hardware is a large and growing share of the devices in people's homes, built on reference designs that are copied across unrelated product lines with the security posture intact — meaning absent. The chipsets in this device, Beken and TXW parts, appear across a wide range of consumer electronics; the design failures documented here are not specific to ear cameras, and Table 6 shows the same four failure classes recurring from fifteen-dollar DVRs up through FDA-recalled implanted medical devices.

What makes this particular case worth publishing is the gap between the sensitivity of the data and the effort spent protecting it. The product ships an on-device neural network for image processing — someone spent real engineering time on the imaging pipeline. The video that pipeline produces goes out over the air in the clear, protected by a password printed in every beacon frame. That is not a resource constraint; it is a priorities problem, and the predictable outcome when no party in a supply chain — OEM, brand, reseller, or app developer — is accountable for security, and no regulation currently forces the question for a device in this category.

11 Conclusion

Four independently unremarkable IoT defects, chained, let anyone within radio range of a Softish C6 watch its video feed, impersonate it to its own companion app, and push it unsigned firmware — with a total interaction cost, at the worst step, of one tap. The device's threat model, to the extent one exists, rests entirely on the assumption that nobody nearby is listening to public radio broadcasts. That assumption fails for a fifteen-dollar Wi-Fi adapter, and it fails for a device that is, by design, pointed inside a human body.

Acknowledgments

I thank CISA for coordination in the continued absence of a responsive vendor security contact.

References

[1] CISA. CSAF Advisory VA-26-251-01. github.com/cisagov/CSAF

- [2] Krebs, B. "Source Code for IoT Botnet 'Mirai' Released." *Krebs on Security*, 2016. krebsonsecurity.com
- [3] Arlo. "Arlo Wi-Fi Default Password Security Vulnerability." Support advisory. arlo.com
- [4] "Patch Now: TP-Link Archer NX Routers Vulnerable to Firmware Takeover." *Security Affairs*, 2025. CVE-2025-15517. securityaffairs.com
- [5] "Wyze Cam Flaw Lets Hackers Remotely Access Your Saved Videos." *BleepingComputer*. CVE-2019-9564. bleepingcomputer.com
- [6] CISA. ICSA-15-125-01B — Hospira Infusion Pump Vulnerabilities. cisa.gov
- [7] "St. Jude Medical Recalls 465,000 Pacemakers Over Security Vulnerabilities." *SecurityWeek*, 2017. securityweek.com
- [8] NIST. IR 8259A — IoT Device Cybersecurity Capability Core Baseline. pages.nist.gov
- [9] NIST. IR 8425 — technical baseline for the FCC Cyber Trust Mark. fcc.gov/CyberTrustMark
- [10] CISA. IoT Acquisition Guidance for Organizations. cisa.gov (PDF)